

1 인공지능과 머신러닝 학습에 유용한 추가 확률분포

1 개요

베르누이분포, 이항분포, 다항분포, 균일분포, 정규분포, 베타분포, 감마분포, t분포, 카이제곱분포, F분포, 디리클레분포는 이미 학습하였다.

본 문서에서는 머신러닝과 인공지능 학습에서 추가로 자주 사용되는 확률분포를 다룬다.

분포	SciPy 클래스	주요 활용
포아송분포	<code>stats.poisson</code>	카운트 데이터, 이벤트 발생 횟수
음이항분포	<code>stats.nbinom</code>	과산포 카운트 데이터
지수분포	<code>stats.expon</code>	대기시간, 생존시간, 이벤트 간격
로그정규분포	<code>stats.lognorm</code>	양수형 왜도 데이터
라플라스분포	<code>stats.laplace</code>	L1 손실, 이상치에 강한 오차 모델
와이블분포	<code>stats.weibull_min</code>	고장시간, 생존분석, 신뢰성 분석
다변수 정규분포	<code>stats.multivariate_normal</code>	다차원 특성, GMM, 베이지 모델

2 공통 Python 준비 코드

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)
```

3 포아송분포

3.1 활용 방법

포아송분포는 일정한 시간 또는 공간 안에서 어떤 사건이 몇 번 발생하는지를 모델링한다. 예를 들어 웹사이트의 분당 방문 수, 광고 클릭 수, 서버 요청 수, 센서 이벤트 수 등을 표현할 수 있다.

머신러닝에서는 카운트 데이터 예측, 이상 탐지, 트래픽 분석, 자연어 처리의 단어 출현 빈도 모델링 등에 활용된다.

3.2 PMF

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

여기서 λ 는 단위 구간에서 평균적으로 발생하는 사건 수이다.

3.3 PMF 시각화 및 시뮬레이션

```
lam = 4
x = np.arange(0, 15)

pmf = stats.poisson.pmf(x, mu=lam)

plt.figure()
plt.bar(x, pmf)
plt.title("Poisson PMF")
plt.xlabel("x")
plt.ylabel("P(X=x)")
plt.show()

samples = stats.poisson.rvs(mu=lam, size=10000)

plt.figure()
plt.hist(samples, bins=np.arange(0, 16)-0.5, density=True)
plt.title("Poisson Simulation")
plt.xlabel("x")
plt.ylabel("Relative Frequency")
plt.show()
```

3.4 머신러닝 활용 예제: 웹 요청 수 이상 탐지

```
# 정상 상태에서 분당 평균 요청 수가 20이라고 가정
lam = 20

# 관측된 분당 요청 수
observed_requests = np.array([18, 21, 19, 22, 20, 35, 17, 90])

# 각 관측값 이상이 발생할 확률:  $P(X \geq x)$ 
p_values = 1 - stats.poisson.cdf(observed_requests - 1, mu=lam)

for x, p in zip(observed_requests, p_values):
    print(f"request={x}, p-value={p:.6f}")

# p-value가 매우 작으면 이상치로 판단
anomalies = observed_requests[p_values < 0.01]
print("Anomalies:", anomalies)
```

4 음이항분포

4.1 활용 방법

음이항분포는 카운트 데이터의 분산이 평균보다 큰 경우에 유용하다. 포아송분포는 평균과 분산이 같다는 가정을 갖지만, 실제 데이터에서는 분산이 더 큰 경우가 많다. 이를 과산포라고 한다.

예를 들어 사용자별 클릭 수, 고객별 구매 횟수, 문서별 특정 단어 등장 횟수 등은 음이항분포로 더 잘 설명될 수 있다.

4.2 PMF

$$P(X = k) = \binom{k+r-1}{k} (1-p)^k p^r, \quad k = 0, 1, 2, \dots$$

여기서 r 은 성공 횟수, p 는 성공 확률이다. SciPy의 `nbinom`에서 X 는 r 번 성공하기 전까지의 실패 횟수를 의미한다.

4.3 PMF 시각화 및 시뮬레이션

```
r, p = 5, 0.4
x = np.arange(0, 30)

pmf = stats.nbinom.pmf(x, n=r, p=p)

plt.figure()
plt.bar(x, pmf)
plt.title("Negative Binomial PMF")
plt.xlabel("x")
plt.ylabel("P(X=x)")
plt.show()

samples = stats.nbinom.rvs(n=r, p=p, size=10000)

plt.figure()
plt.hist(samples, bins=30, density=True)
plt.title("Negative Binomial Simulation")
plt.xlabel("x")
plt.ylabel("Relative Frequency")
plt.show()
```

4.4 머신러닝 활용 예제: 과산포 카운트 데이터 확인

```
# 사용자별 클릭 수 데이터라고 가정
clicks = stats.nbinom.rvs(n=3, p=0.3, size=1000)

mean_clicks = np.mean(clicks)
var_clicks = np.var(clicks)

print("Mean:", mean_clicks)
print("Variance:", var_clicks)

if var_clicks > mean_clicks:
    print("과산포가 존재하므로 포아송보다 음이항분포가 적합할 수 있음")
```


5 지수분포

5.1 활용 방법

지수분포는 어떤 사건이 발생할 때까지의 대기시간을 모델링한다. 서버 요청 간격, 고객 이탈까지의 시간, 장비 고장까지의 시간, 강화학습에서 이벤트 발생 간격 등을 표현할 수 있다.

5.2 PDF

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

여기서 λ 는 사건 발생률이다. SciPy에서는 `scale=1/λ`를 사용한다.

5.3 PDF 시각화 및 시뮬레이션

```
lam = 1.5
x = np.linspace(0, 5, 300)

pdf = stats.expon.pdf(x, scale=1/lam)

plt.figure()
plt.plot(x, pdf)
plt.title("Exponential PDF")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()

samples = stats.expon.rvs(scale=1/lam, size=10000)

plt.figure()
plt.hist(samples, bins=50, density=True)
plt.plot(x, pdf)
plt.title("Exponential Simulation")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()
```

5.4 머신러닝 활용 예제: 고객 이탈 시간 모델링

```
# 평균적으로 10일 후 이탈한다고 가정
mean_time = 10
lam = 1 / mean_time

# 5일 이내 이탈할 확률
prob_churn_5days = stats.expon.cdf(5, scale=1/lam)

# 20일 이상 유지될 확률
prob_survive_20days = 1 - stats.expon.cdf(20, scale=1/lam)

print("P(churn <= 5 days):", prob_churn_5days)
print("P(survive >= 20 days):", prob_survive_20days)
```

6 로그정규분포

6.1 활용 방법

로그정규분포는 로그를 취한 값이 정규분포를 따르는 양수 확률변수를 모델링한다. 소득, 매출, 파일 크기, 처리 시간, 응답 시간, 가격 데이터처럼 0보다 크고 오른쪽 꼬리가 긴 데이터에 적합하다.

6.2 PDF

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left[-\frac{(\ln x - \mu)^2}{2\sigma^2}\right], \quad x > 0$$

6.3 PDF 시각화 및 시뮬레이션

```
sigma, mu = 0.6, 0.0
x = np.linspace(0.01, 6, 400)

pdf = stats.lognorm.pdf(x, s=sigma, scale=np.exp(mu))

plt.figure()
plt.plot(x, pdf)
plt.title("Lognormal PDF")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()

samples = stats.lognorm.rvs(s=sigma, scale=np.exp(mu), size=10000)

plt.figure()
plt.hist(samples, bins=60, density=True)
plt.plot(x, pdf)
plt.title("Lognormal Simulation")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()
```

6.4 머신러닝 활용 예제: 응답 시간 데이터의 로그 변환

```
# 웹 서비스 응답 시간이 로그정규분포를 따른다고 가정
response_time = stats.lognorm.rvs(s=0.7, scale=np.exp(1.0), size=1000)

# 원 데이터는 오른쪽 꼬리가 김
print("Original mean:", np.mean(response_time))
print("Original median:", np.median(response_time))

# 로그 변환 후 정규분포에 가까워짐
log_response_time = np.log(response_time)

print("Log-transformed mean:", np.mean(log_response_time))
print("Log-transformed std:", np.std(log_response_time))

plt.figure()
plt.hist(response_time, bins=50, density=True)
plt.title("Original Response Time")
plt.show()

plt.figure()
plt.hist(log_response_time, bins=50, density=True)
plt.title("Log-transformed Response Time")
plt.show()
```

7 라플라스분포

7.1 활용 방법

라플라스분포는 정규분포보다 중심이 뾰족하고 꼬리가 두껍다. 이상치가 존재하는 오차 데이터를 모델링할 때 유용하다.

머신러닝에서는 L1 손실, Lasso 회귀, 희소성 유도, 강건한 회귀 모델과 관련이 깊다.

7.2 PDF

$$f(x) = \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right)$$

여기서 μ 는 위치 모수, b 는 스케일 모수이다.

7.3 PDF 시각화 및 시뮬레이션

```
mu, b = 0, 1
x = np.linspace(-8, 8, 400)

pdf = stats.laplace.pdf(x, loc=mu, scale=b)

plt.figure()
plt.plot(x, pdf)
plt.title("Laplace PDF")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()

samples = stats.laplace.rvs(loc=mu, scale=b, size=10000)

plt.figure()
plt.hist(samples, bins=60, density=True)
plt.plot(x, pdf)
plt.title("Laplace Simulation")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()
```


7.4 머신러닝 활용 예제: L1 손실과 L2 손실 비교

```
y_true = np.array([3, 3, 3, 3, 3])
y_pred = np.array([2.8, 3.1, 3.0, 2.9, 10.0])

errors = y_true - y_pred

l1_loss = np.mean(np.abs(errors))
l2_loss = np.mean(errors ** 2)

print("Errors:", errors)
print("L1 loss:", l1_loss)
print("L2 loss:", l2_loss)
```

큰 이상치가 있을 때 L2 손실은 훨씬 크게 증가한다.
라플라스 오차 가정은 L1 손실과 연결된다.

8 와이블분포

8.1 활용 방법

와이블분포는 고장시간, 생존시간, 수명 데이터를 모델링할 때 자주 사용된다. 위험률이 시간이 지남에 따라 증가하거나 감소하는 상황을 표현할 수 있다.

예측 유지보수, 장비 고장 예측, 의료 생존분석, 제품 신뢰성 분석 등에 활용된다.

8.2 PDF

$$f(x) = \frac{k}{\lambda} \left(\frac{x}{\lambda}\right)^{k-1} \exp\left[-\left(\frac{x}{\lambda}\right)^k\right], \quad x \geq 0$$

여기서 k 는 형상 모수, λ 는 스케일 모수이다.

8.3 PDF 시각화 및 시뮬레이션

```

k, lam = 1.8, 2.0
x = np.linspace(0, 6, 400)

pdf = stats.weibull_min.pdf(x, c=k, scale=lam)

plt.figure()
plt.plot(x, pdf)
plt.title("Weibull PDF")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()

samples = stats.weibull_min.rvs(c=k, scale=lam, size=10000)

plt.figure()
plt.hist(samples, bins=50, density=True)
plt.plot(x, pdf)
plt.title("Weibull Simulation")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()

```

8.4 머신러닝 활용 예제: 장비 고장 확률 계산

```

k, lam = 1.8, 1000 # 평균적인 수명 규모를 1000시간으로 가정

# 500시간 이내 고장날 확률
p_fail_500 = stats.weibull_min.cdf(500, c=k, scale=lam)

# 1500시간 이상 버틸 확률
p_survive_1500 = 1 - stats.weibull_min.cdf(1500, c=k, scale=lam)

print("P(failure <= 500 hours):", p_fail_500)
print("P(survival >= 1500 hours):", p_survive_1500)

# 예측 유지보수에서는 특정 시간 이전에 고장날 확률이 임계값보다 크면
# 사전 점검 또는 교체 대상으로 분류할 수 있다.

```

9 다변수 정규분포

9.1 활용 방법

다변수 정규분포는 여러 개의 연속형 특성이 동시에 정규분포 구조를 가질 때 사용된다. 각 변수의 평균뿐 아니라 변수들 사이의 공분산도 함께 모델링한다.

머신러닝에서는 다음과 같은 곳에서 중요하다.

- 가우시안 혼합 모델(Gaussian Mixture Model)
- 베이지 분류기
- 이상 탐지
- 잠재변수 모델
- 칼만 필터
- 다차원 특성 시뮬레이션

9.2 PDF

d 차원 확률변수 $\mathbf{x}$ 가 평균벡터 $\boldsymbol{\mu}$ 와 공분산행렬 Σ 를 갖는 다변수 정규분포를 따른다면 다음과 같이 쓴다.

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

확률밀도함수는 다음과 같다.

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right]$$

9.3 PDF 등고선 시각화 및 시뮬레이션

```

mean = np.array([0, 0])
cov = np.array([[1.0, 0.8],
                 [0.8, 1.5]])

rv = stats.multivariate_normal(mean=mean, cov=cov)

x = np.linspace(-4, 4, 100)
y = np.linspace(-4, 4, 100)
X, Y = np.meshgrid(x, y)

pos = np.dstack((X, Y))
Z = rv.pdf(pos)

plt.figure()
plt.contour(X, Y, Z)
plt.title("Multivariate Normal PDF Contour")
plt.xlabel("x1")
plt.ylabel("x2")
plt.axis("equal")
plt.show()

samples = rv.rvs(size=2000)

plt.figure()
plt.scatter(samples[:, 0], samples[:, 1], s=5, alpha=0.5)
plt.title("Multivariate Normal Simulation")
plt.xlabel("x1")
plt.ylabel("x2")
plt.axis("equal")
plt.show()

```

9.4 머신러닝 활용 예제: 마할라노비스 거리를 이용한 이상 탐지

다변수 정규분포에서는 단순한 유클리드 거리보다 공분산 구조를 고려한 마할라노비스 거리가 중요하다.

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

```
mean = np.array([0, 0])
cov = np.array([[1.0, 0.8],
                [0.8, 1.5]])

normal_data = stats.multivariate_normal.rvs(
    mean=mean,
    cov=cov,
    size=500
)

# 이상치 후보
test_points = np.array([
    [0.5, 0.2],
    [2.0, 2.5],
    [5.0, -4.0]
])

cov_inv = np.linalg.inv(cov)

def mahalanobis_distance(x, mean, cov_inv):
    diff = x - mean
    return np.sqrt(diff.T @ cov_inv @ diff)

distances = np.array([
    mahalanobis_distance(x, mean, cov_inv)
    for x in test_points
])

print("Mahalanobis distances:", distances)

# 2차원 정규분포에서 거리 제곱은 대략 자유도 2인 카이제곱분포와 연결된다.
threshold = np.sqrt(stats.chi2.ppf(0.99, df=2))

print("Threshold:", threshold)
print("Anomaly flags:", distances > threshold)
```